A Practical Guide to Running VASP on High Performance Computing (HPC) Systems

Dr. Md Delowar Hossain

Material Science and Engineering Department

College of Chemicals and Materials

KING FAHD UNIVERSITY OF PETROLEUM AND MINERAL

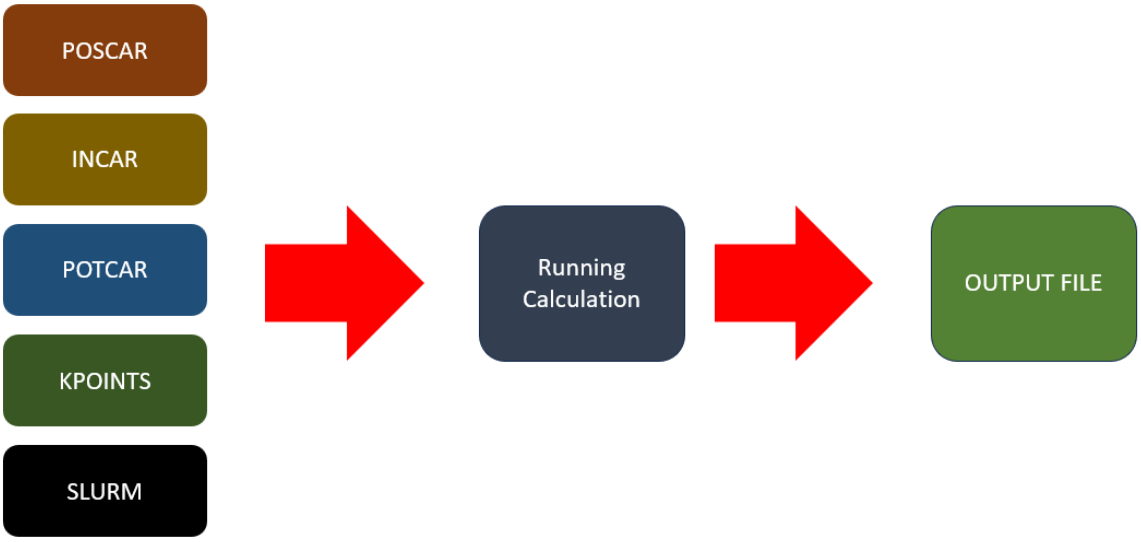
DHAHRAN, SAUDI ARABIA

I. Introduction

The Vienna *Ab-initio* Simulation Package (VASP) is a software package used for first-principles simulations of materials at the atomic scale. It is commonly applied to calculate electronic structures and optimize atomic geometries based on quantum mechanical principles. VASP mainly operates within the framework of Density Functional Theory (DFT) and uses a plane-wave basis set with the Projector Augmented-Wave (PAW) method. It supports structural optimization, molecular dynamics, and advanced electronic property calculations, making it well suited for periodic systems such as crystals. This guideline focuses on the practical use of VASP on *High Performance Computing* (HPC) systems, including workflow, input preparation, job submission, and basic result analysis.

II. Input File for VASP

VASP calculations require four main input files: **INCAR, POSCAR, POTCAR, and KPOINTS**. Each file controls a specific part of the simulation.



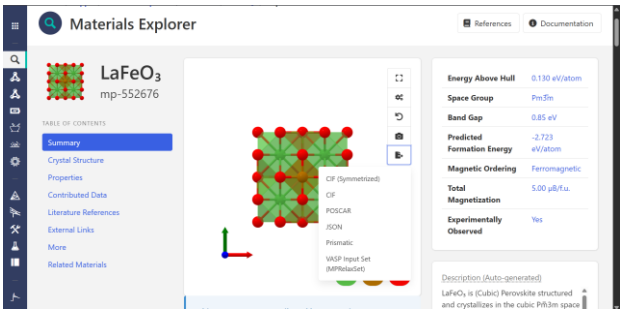
1) POSCAR

The POSCAR file is a mandatory VASP input file. It is a plain text file and contains at least the lattice geometry and the ionic positions. Optionally, also starting velocities for a molecular-dynamics simulation can be provided here. This file shares its format with VASP output file CONTCAR. That may contain an additional section with predictor-corrector coordinates necessary for restarting molecular-dynamics runs. Creating a POSCAR file is often

the starting point of VASP-supported research. It can be written manually or obtained from various online materials and crystallographic databases providing a download in the POSCAR file format. POSCAR files can be visualized using a variety of softwares, including VESTA, Jmol, OVITO, and the Atomic Simulation Environment (ASE).

Example POSCAR

Open material project https://next-gen.materialsproject.org/, search $LaFeO_3$ and download POSCAR file and now you have POSCAR file




```
La Fe O
 1.0000000000000000
    3.9600000000000000    0.0000000000000000    0.0000000000000000
    0.0000000000000000    3.9600000000000000    0.0000000000000000
    0.0000000000000000    0.0000000000000000    3.9600000000000000
 La  Fe  O
   1   1   3
Cartesian
  0.0000000000000000  0.0000000000000000  0.0000000000000000
  1.9800000000000000  1.9800000000000000  1.9800000000000000
  1.9800000000000000  1.9800000000000000  0.0000000000000000
  1.9800000000000000  0.0000000000000000  1.9800000000000000
  0.0                       0000  1.9800000000000000
```

2) INCAR

The INCAR file is the central input file of VASP, which determines what to do and how to do it. The INCAR tags specified in the INCAR file select the algorithms and set the parameters that VASP uses during the calculation. VASP will use reasonable default values, which we recommend using when unsure. Yet, the settings in the INCAR file are the main source of errors and false results, so we suggest carefully checking the meaning of the set INCAR tags. Regarding the format, each statement consists of the name of a tag, the equal sign =, and the values assigned to the tag (tag = values). For instance, an INCAR file for a density-functional-theory calculation might read.

Example INCAR

```
#Exchange–Correlation & Accuracy
GGA   = PE        # Use GGA-PBE exchange–correlation functional (Good for metal, alloy or surface)
PREC  = Normal    # Numerical precision level (Normal is a good accuracy–cost balance)
ENCUT = 500       # Plane-wave energy cutoff (eV), must exceed POTCAR recommendation (1.5 x max. encut in the POTCAR)

#Electronic SCF Control
EDIFF = 1E-5        # Electronic self-consistency convergence criterion (eV) -> "the total change in electronic energy between two consecutive iterations is less than 1E-5 eV"
NELM  = 500         # Maximum number of electronic SCF iterations -> "If convergence is reached before 250 iterations, the calculation stops automatically"
ALGO  = Normal    # Stable SCF algorithm for metals and surfaces

KPAR = 4      # Number of k-point parallelization groups (parallelizes over k-points)
NPAR = 16     # Number of bands/FFT parallelization within each k-point group
#Charge & Spin Mixing Parameters (If you want to use)
AMIX     = 0.10   # Linear mixing parameter for charge density
BMIX     = 0.0001 # Kerker mixing parameter for charge density
AMIX_MAG = 0.10   # Linear mixing parameter for spin density
BMIX_MAG = 0.0001 # Kerker mixing parameter for spin density
LMAXMIX  = 6      # Maximum angular momentum for charge mixing (d-electrons)

#PAW and Projection Settings
LASPH = .TRUE.    # Include non-spherical contributions in PAW spheres
LREAL = Auto      # Automatic real-space projection for large systems

#Smearing
ISMEAR = 0       # # Gaussian smearing method, suitable for semiconductors and insulators (order 0), Methfessel–Paxton smearing (order 1) for metals
SIGMA  = 0.05     # Smearing width (eV) to aid SCF convergence

#Ionic Relaxation (Geometry Optimization)
IBRION = 2        # Conjugate-gradient algorithm for ionic relaxation
NSW    = 500      # Maximum number of ionic relaxation steps "Sets a limit on how many times the position of an atom can be updated."
ISIF   = 2        # Relax atomic positions only (cell shape fixed)
EDIFFG = -0.05    # Force convergence criterion (eV/A) -> "Relaxation stops when the force is maximum on all atoms < 0.05 eV/A"

#Spin Polarization
ISPIN  = 2        # Enable spin-polarized DFT calculations
MAGMOM = 1*0.01 1*5 3*-0.01  # (La-Fe-O) Initial magnetic moments for each atom (μB)

#DFT+U (Strongly Correlated d-electrons)
LDAU      = .TRUE. # Enable DFT+U correction
LDAUTYPE  = 2      # Dudarev scheme (Ueff = U − J)
```

```
LDAUPRINT = 2      # Print DFT+U related information
LDAUL     = -1 2 -1 # Apply U to d-orbitals of transition metal only
LDAUU     = 0.0 4.3 0.0 # Hubbard U values (eV)
LDAUJ     = 0.0 0.0 0.0 # Exchange parameter J (set to zero)

#Dipole Correction (Slab Models)
LDIPOL = .TRUE.   # Enable dipole correction for asymmetric slabs
IDIPOL = 3        # Apply dipole correction along z-direction
DIPOL  = 0.5 0.5 0.5 # Reference point for dipole correction

#van der Waals Interactions
IVDW = 12          # Enables DFT-D3 van der Waals correction (Grimme), including three-body terms

#Output Control
LORBIT = 11       # Enable projected DOS and magnetic moment output
LWAVE  = .FALSE.  # Do not write WAVECAR (save disk space)
LCHARG = .FALSE.  # Do not write CHGCAR
LAECHG = .FALSE.  # Do not write all-electron charge density
LVTOT  = .FALSE.  # Do not write local electrostatic potential

#Restart & Charge Density
ISTART = 0        # Start calculation from scratch
ICHARG = 2        # Initial charge density from atomic superposition
```

3) POTCAR

The POTCAR file is a mandatory VASP input file that contains the pseudopotentials for each atomic species, listed in the same order as the elements defined in the POSCAR file. It is created by concatenating the POTCAR files of individual elements.

Example POTCAR for LaFeO3

```
PAW_PBE La 06Sep2000
   11.0000000000000
 parameters from PSCTR are:
….
End of Dataset

PAW_PBE Fe 06Sep2000
   8.00000000000000
 parameters from PSCTR are:
….
End of Dataset

PAW_PBE O 08Apr2002
   6.00000000000000
 parameters from PSCTR are:
```

```
….
End of Dataset
```

4) KPOINTS

The KPOINTS file specifies the Bloch vectors (k points) used to sample the Brillouin zone. Converging this sampling is one of the essential tasks in many calculations concerning the electronic minimization.

Example KPOINTS

```
KPOINTS
0
Gamma
5 5 5
0 0 0
```

5) SLURM

On HPC clusters, VASP calculations are usually run using a job scheduler such as SLURM (Simple Linux Utility for Resource Management). A SLURM submission file (for example job.slurm, vasp.slurm, or submit.sh) is a plain text script that defines the job name, requested resources (nodes, cores, and time limit), output and error files, required software modules (such as VASP, MPI, and Intel MKL), and the execution command, typically using srun. This script ensures that the calculation runs with the correct environment and parallel settings and can be reproduced on the cluster.

Example SLURM

```
#!/usr/bin/env bash
#SBATCH -J FILE_NAME
#SBATCH -p all
#SBATCH -N 1
#SBATCH --ntasks-per-node=64
#SBATCH --time=7-00:00:00
#SBATCH --hint=nomultithread
#SBATCH --output=slurm-%j.out
#SBATCH --error=slurm-%j.err

set -euo pipefail
echo "Job started on $(date)"
echo "Job ID: $SLURM_JOB_ID"
```

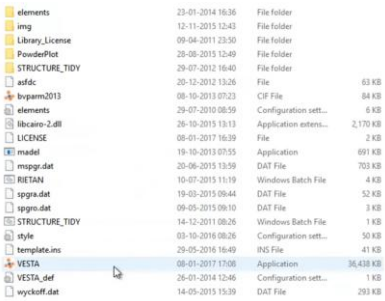
```
echo "PWD: $(pwd)"

module purge
module load vasp/6.5.0-cpu-ompi-intel1api
module load intel-oneapi-mkl/2025.2
module load intel-oneapi/2025.2.lua
module load conda/25.08
module load ase/3.23.0

ulimit -s unlimited
export OMP_NUM_THREADS=1
export VASP_COMMAND="srun --mpi=pmix -n ${SLURM_NTASKS} vasp_std"
#export VASP_COMMAND="srun -n ${SLURM_NTASKS} vasp_std"
export VASP_PP_PATH=/home/g202425580
echo "VASP_COMMAND=$VASP_COMMAND"
echo "VASP_PP_PATH=$VASP_PP_PATH"

python -u ./run3.py
```

III. Download and Install VESTA

For visualization or check the structure

1) Download VESTA for your device from this link https://jp-minerals.org/vesta/en/download.html

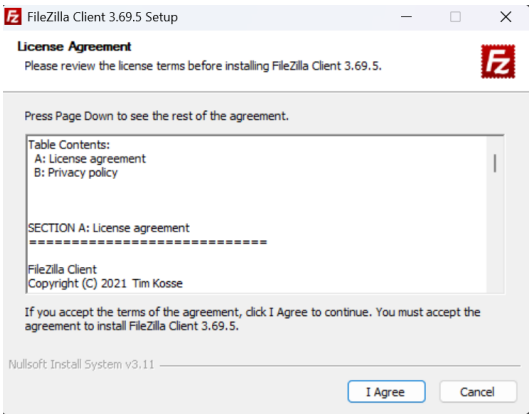
2) Extract the downloaded zip file and then run VESTA.exe



IV. Download and Install FileZilla

For transfer file from our storage to server

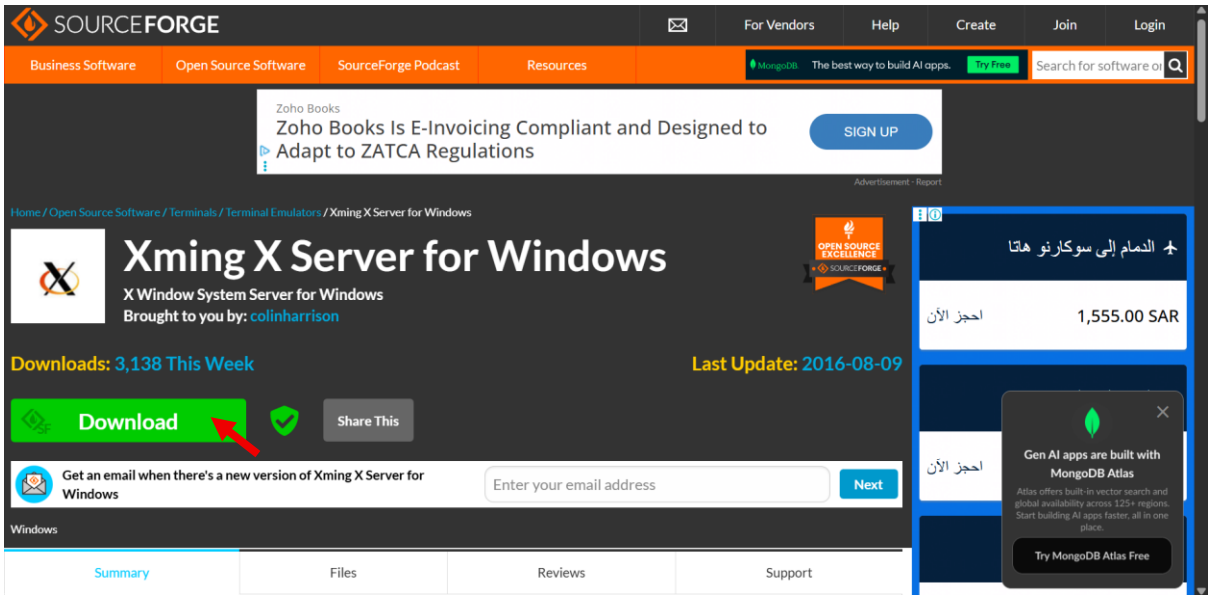
Download FileZilla from https://filezilla-project.org/download.php?platform=win64 and install it on your computer until finish



V. Download and Install Xming (Windows)

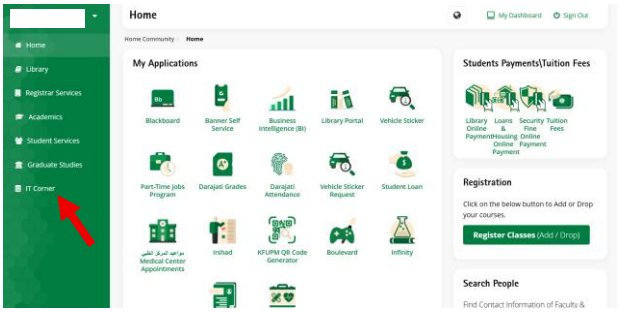
For open ASE GUI in the server

Download Xming from https://sourceforge.net/projects/xming/ and install it on your computer until finish

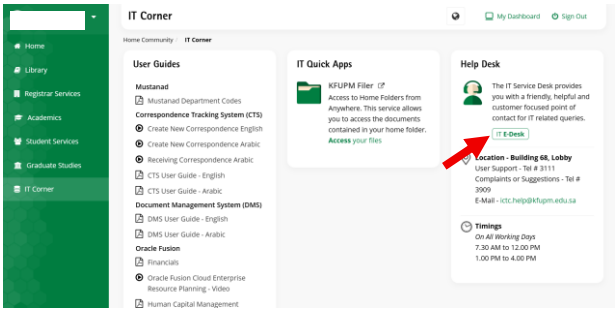


VI. Request HPC Account

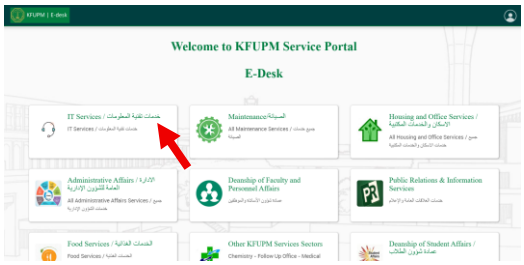
1) Log in to the KFUPM portal and click **IT Corner** from the left-side menu.



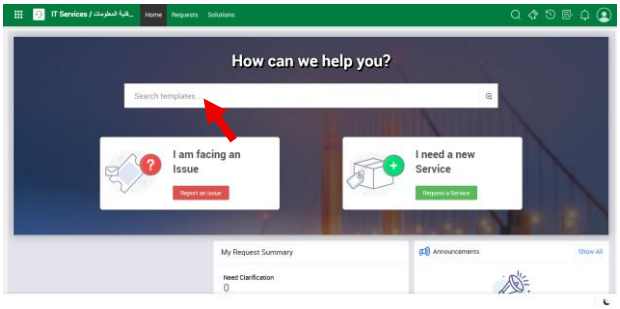
2) Click **IT E-Desk** from the right-side menu.



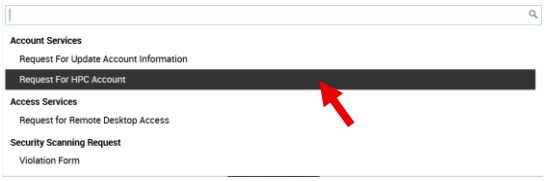
3) Click **IT Services / خدمات تقنية المعلومات**.

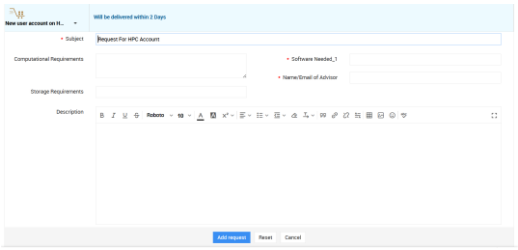


4) Click **Search templates …**

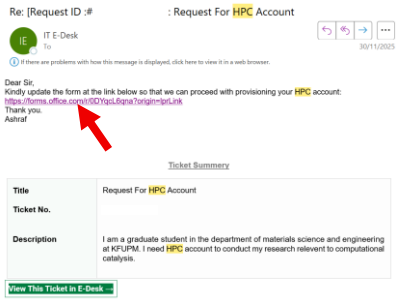


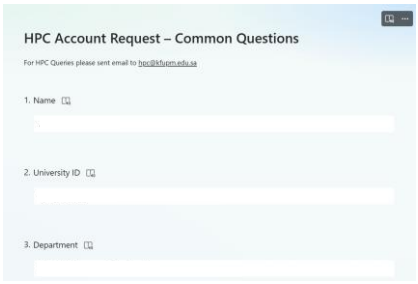
5) Click **Request for HPC Account** and fill in the required information in the HPC account request form and click **Add request** to submit**.**



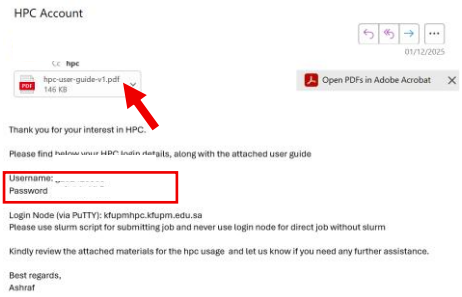


6) After submitting your request, you will receive an email containing a link to the HPC account form; please complete the form according to your research requirements.



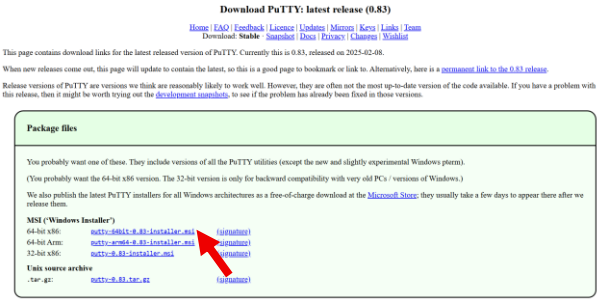


7) After approval, you will receive a confirmation email containing your default HPC account details, including your **username**, **password**, and the attached **HPC user guide.**

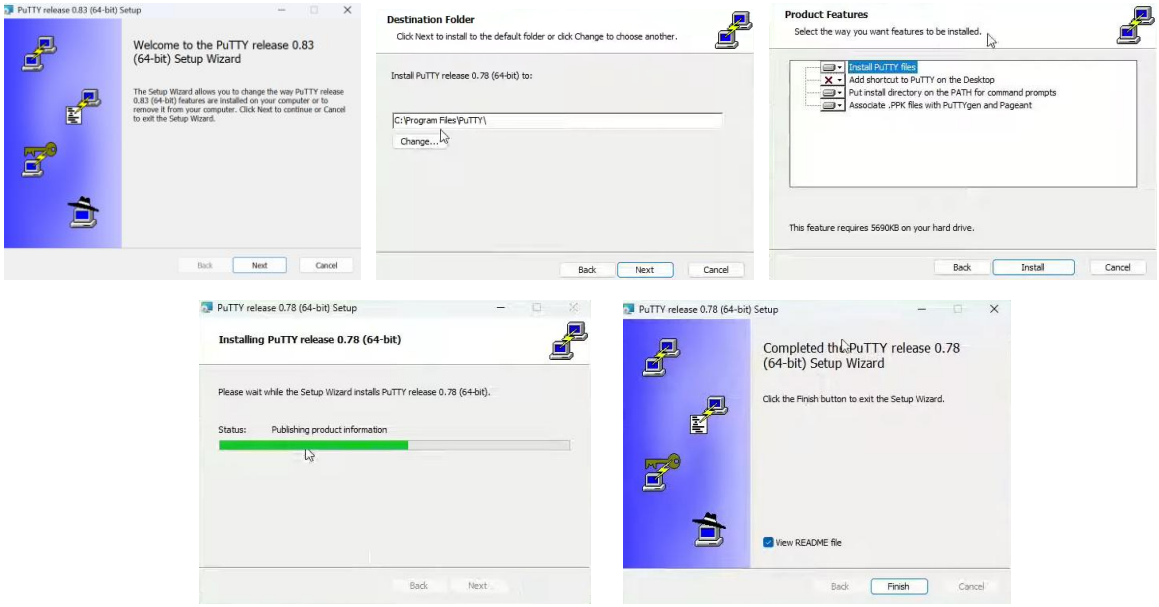


VII. Download and Install PuTTY for Access the HPC Server

1) To access the HPC server on Windows, download PuTTY from https://www.chiark.greenend.org.uk/~sgtatham/putty/latest.html and select the 64-bit x86 installer.

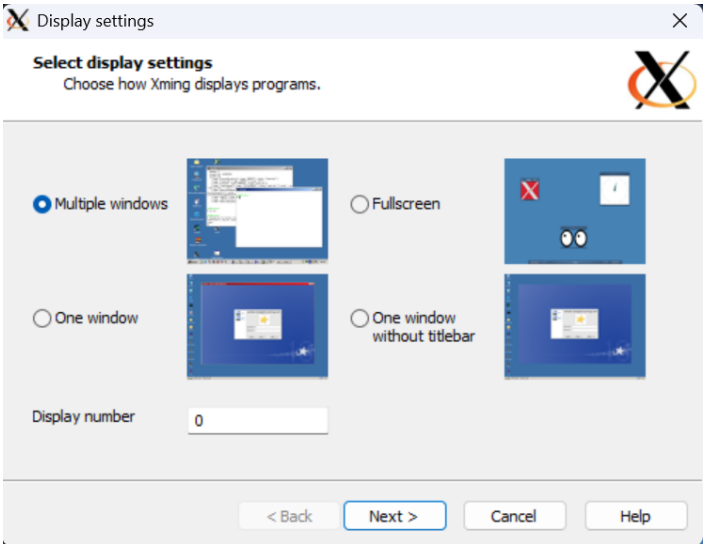


2) Install PuTTY until **Finish.**

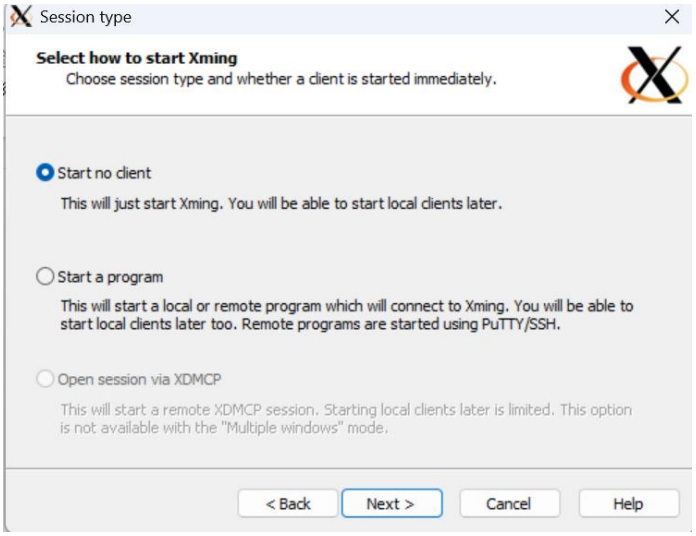


VIII. XMING preparation

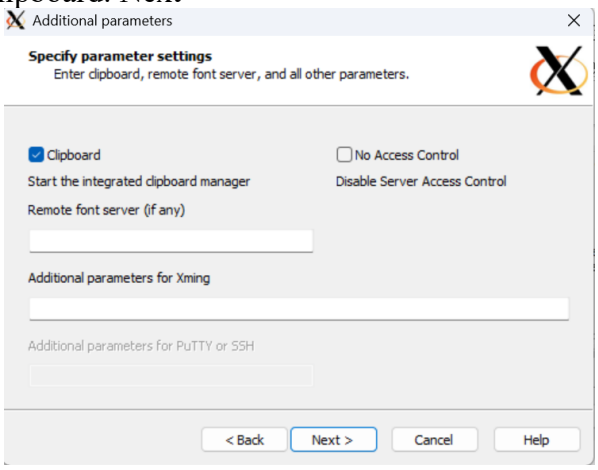
1) After install XMING we have two icon, XMING and XLAUNCH. The, open XLAUNCH. Click Multiple windows and Display Number is 0. Next



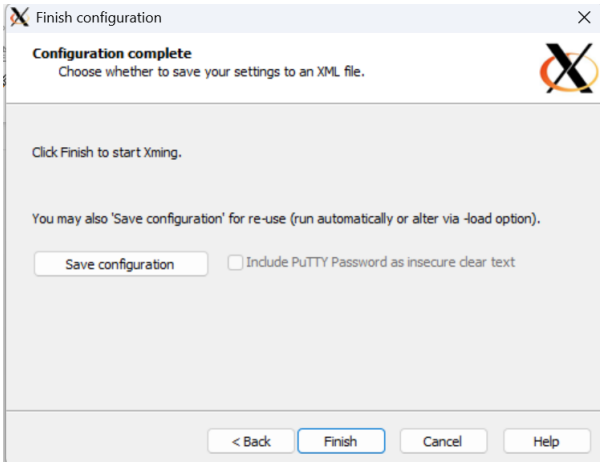
2) Click start no client. Next



3) Check the mark Clipboard. Next

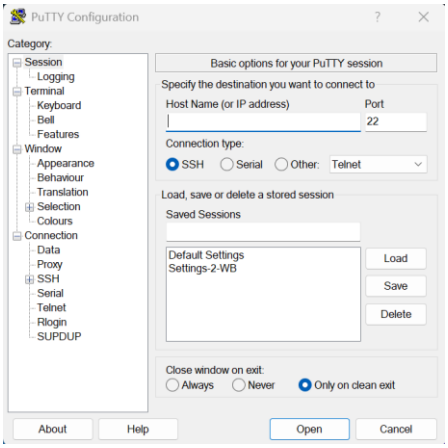


4) Finish

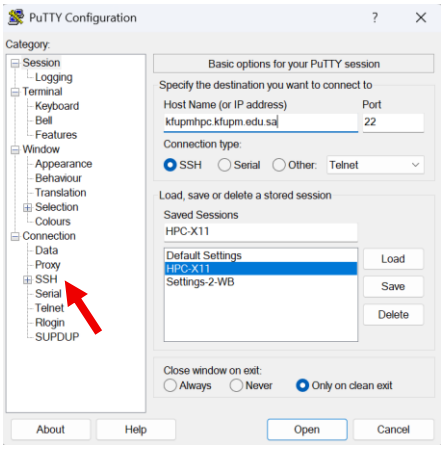


IX. PuTTY preparation and change the password

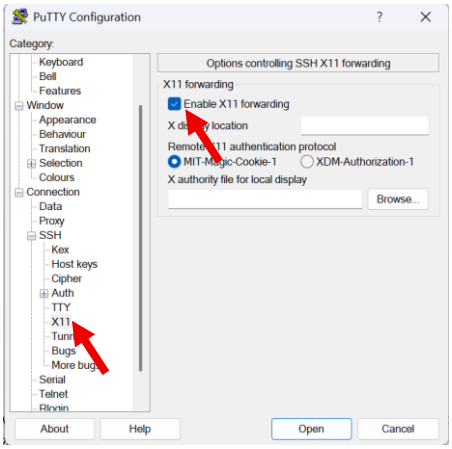
1) Open PuTTY, and enter:
   Host Name : kfupmhpc.kfupm.edu.sa
   Port : 22
   Connection type : SSH



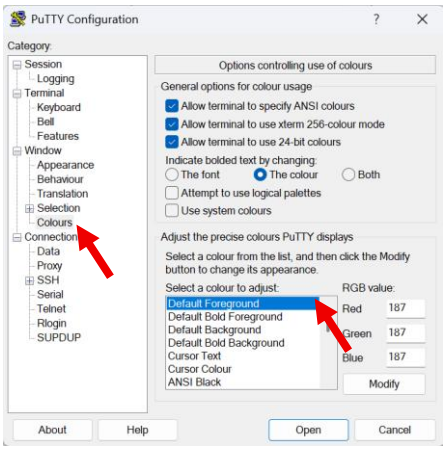
2) In **Saved Sessions**, type your session name, for example "HPC-X11". Then click **SSH**.



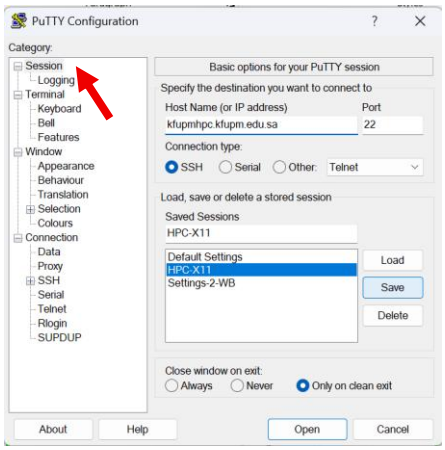
3) Click **X11** then check the **Enable X11 forwarding** section (***This is important for open ASE GUI in server)***



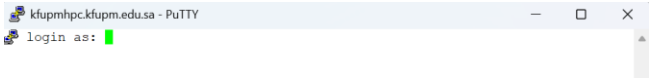
4) This is optional if you want to change the appearance of the PuTTY terminal. **Default Foreground** is the text color in the terminal, for example, if you want to change it to black, and **Default Background** is the background color, for example, white.

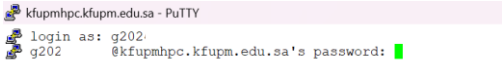


5) Go back to **Session** then **save** and **open**



6) Login with your account. Login as: **g202xxxxxx** and input your **default password** (*the password is not shown when you type it*), you can **copy** your password to **paste** it, just **right click** and **enter**.





7) After logging in successfully, type the following command and press **Enter**:

```
passwd
```

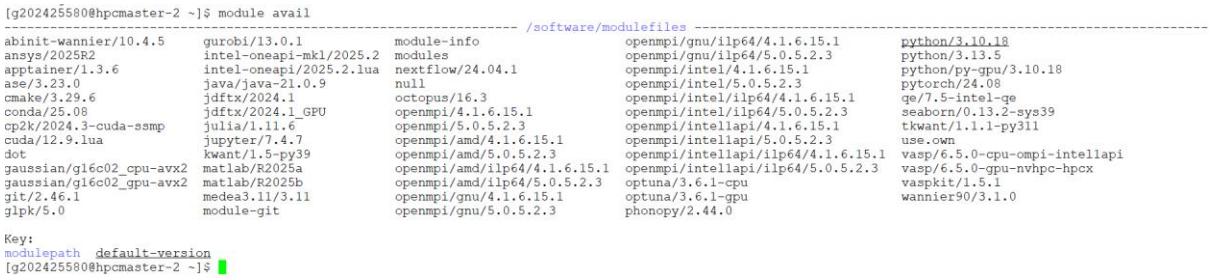
8) When prompted, enter your **current (default) password**, enter your **new password**, then press **Enter** and Re-enter the **new password** to confirm:
   Current password:
   New password:
   Retype new password:

9) If the password change is successful, you will see the message:

   passwd: password updated successfully

X. Running VASP on HPC

1) After logging in and changing the password, check the available modules on your account using the "**module avail"** command. Make sure you have the **ase** and **python** modules.



```
[g202425580@hpcmaster-2 ~]$ module avail
------------------------------------------------------------------ /software/modulefiles ------------------------------------------------------------------
abinit-wannier/10.4.5     gurobi/13.0.1           module-info                 openmpi/gnu/ilp64/4.1.6.15.1       python/3.10.18
ansys/2025R2              intel-oneapi-mkl/2025.2 modules                     openmpi/gnu/ilp64/5.0.5.2.3        python/3.13.5
apptainer/1.3.6           intel-oneapi/2025.2.lua nextflow/24.04.1            openmpi/intel/4.1.6.15.1           python/py-gpu/3.10.18
ase/3.23.0                java/java-21.0.9        null                        openmpi/intel/5.0.5.2.3            pytorch/24.08
cmake/3.29.6              jdftx/2024.1            octopus/16.3                openmpi/intel/ilp64/4.1.6.15.1     qe/7.5-intel-qe
conda/25.08               jdftx/2024.1_GPU        openmpi/4.1.6.15.1          openmpi/intel/ilp64/5.0.5.2.3      seaborn/0.13.2-sys39
cp2k/2024.3-cuda-ssmp     julia/1.11.6            openmpi/5.0.5.2.3           openmpi/intellapi/4.1.6.15.1       tkwant/1.1.1-py311
cuda/12.9.lua             jupyter/7.4.7           openmpi/amd/4.1.6.15.1      openmpi/intellapi/5.0.5.2.3        use.own
dot                       kwant/1.5-py39          openmpi/amd/5.0.5.2.3       openmpi/intellapi/ilp64/4.1.6.15.1 vasp/6.5.0-cpu-ompi-intellapi
gaussian/g16c02_cpu-avx2  matlab/R2025a           openmpi/amd/ilp64/4.1.6.15.1 openmpi/intellapi/ilp64/5.0.5.2.3 vasp/6.5.0-gpu-nvhpc-hpcx
gaussian/g16c02_gpu-avx2  matlab/R2025b           openmpi/amd/ilp64/5.0.5.2.3 optuna/3.6.1-cpu                   vaspkit/1.5.1
git/2.46.1                medea3.11/3.11          openmpi/gnu/4.1.6.15.1      optuna/3.6.1-gpu                   wannier90/3.1.0
glpk/5.0                  module-git              openmpi/gnu/5.0.5.2.3       phonopy/2.44.0

Key:
modulepath  default-version
[g202425580@hpcmaster-2 ~]$
```

2) Open the .bashrc file using "**vi ~/.bashrc"** to view or modify shell settings, aliases, and environment variables that are loaded automatically at login. Copy and paste this conditions for easier our work. The way to copy paste here is a little different, first as usual here you just **CTRL+C** in the empty terminal section just **right click** the mouse or you can download bashrc file https://github.com/jahrilnurfauzan/DFT.

```
#alias
alias l="ls -latrh --color"
alias pip="python -m pip"
alias ag='ase gui'
alias cp='cp -v'
alias sc="source ~/.bashrc"
alias me="squeue -u g202425580"
alias rds='cd $RDS/home/'

# >>> conda initialize >>>
# !! Contents within this block are managed by 'conda init' !!
__conda_setup="$('/software/conda/bin/conda' 'shell.bash' 'hook' 2> /dev/null)"
if [ $? -eq 0 ]; then
    eval "$__conda_setup"
else
    if [ -f "/software/conda/etc/profile.d/conda.sh" ]; then
        . "/software/conda/etc/profile.d/conda.sh"
    else
        export PATH="/software/conda/bin:$PATH"
```

```
    fi
fi
unset __conda_setup
# <<< conda initialize <<<

case $- in
  *i*)
    conda activate ase-gui

    # X11 check (info only, no prompt change)
    if [[ -n "$DISPLAY" ]]; then
        echo "[X11 OK] DISPLAY=$DISPLAY"
    else
        echo "[X11 OFF]"
    fi
  ;;
esac
```

Once filled, do this to save the file, press **ESC** then press **SHIFT** and **:** together (this will bring up another command for you at the bottom) then type "**wq!**" to save, if "**q!**" just quit

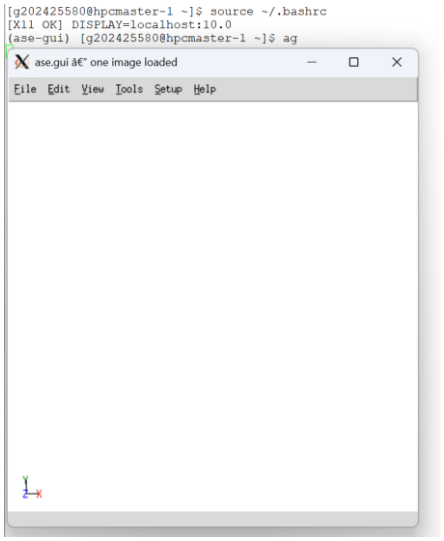
3) Type "**source ~/.bashrc**" to activate your previously created ~/.bashrc. If you see X11 OFF, it means you haven't activated XMING, try to activate XMING and re-login.

```
[g202425580@hpcmaster-2 ~]$ source ~/.bashrc
[X11 OFF]
(ase-gui) [g202425580@hpcmaster-2 ~]$
```

If it is active, you will find the text X11 OK

```
[g202425580@hpcmaster-1 ~]$ source ~/.bashrc
[X11 OK] DISPLAY=localhost:10.0
(ase-gui) [g202425580@hpcmaster-1 ~]$
```

Test with typing "**ag &**" to open ASE GUI.



4) Type "**ls -ltr**" lists the contents of the current directory in long format, sorted by modification time from oldest to newest; in a newly created working directory with no files or folders, the output should be empty.

```
(ase-gui) [g202425580@hpcmaster-1 ~]$ ls -ltr
total 33
-rw-r--r--.   1 g202425580 hpc     0 Dec  7 13:43 input_tmp.in
drwxr-xr-x. 345 g202425580 hpc 32768 Jan 22 15:41 potpaw_PBE
drwxr-xr-x.   5 g202425580 hpc  4096 Feb  2 21:42 project_SrCoO3
[1]+  Done                    ase gui
(ase-gui) [g202425580@hpcmaster-1 ~]$
```

5) Type "**mkdir Name_Directory"** example "**mkdir TRAINING**" which functions to create a directory, then type "**ls -ltr**" again to see it.

```
(ase-gui) [g202425580@hpcmaster-1 ~]$ mkdir TRAINING
(ase-gui) [g202425580@hpcmaster-1 ~]$ ls -ltr
total 33
-rw-r--r--.   1 g202425580 hpc     0 Dec  7 13:43 input_tmp.in
drwxr-xr-x. 345 g202425580 hpc 32768 Jan 22 15:41 potpaw_PBE
drwxr-xr-x.   5 g202425580 hpc  4096 Feb  2 21:42 project_SrCoO3
drwxr-xr-x.   2 g202425580 hpc  4096 Feb 10 22:03 TRAINING
(ase-gui) [g202425580@hpcmaster-1 ~]$
```

6) Open the directory by typing "**cd TRAINING**". It will appear empty because it hasn't been filled with anything.

```
(ase-gui) [g202425580@hpcmaster-1 ~]$ cd TRAINING
(ase-gui) [g202425580@hpcmaster-1 TRAINING]$ ls -ltr
total 0
(ase-gui) [g202425580@hpcmaster-1 TRAINING]$
```

7) Try to make directory again with the name Bulk, "**mkdir Bulk**" and "**ls -ltr"** So now your position is **/home/g202xxxxx/TRAINING/Bulk** (*For the first time we make bulk our structure)* and open Bulk directory "**cd Bulk**"

```
(ase-gui) [g202425580@hpcmaster-1 TRAINING]$ mkdir Bulk
(ase-gui) [g202425580@hpcmaster-1 TRAINING]$ ls -ltr
total 1
drwxr-xr-x. 2 g202425580 hpc 4096 Feb 10 22:10 Bulk
(ase-gui) [g202425580@hpcmaster-1 TRAINING]$
```

8) Make directory again for Low Spin (LS) and High Spin (HS) in the Bulk directory

```
drwxr-xr-x. 2 g202425580 hpc 4096 Feb 10 22:54 LS
drwxr-xr-x. 2 g202425580 hpc 4096 Feb 10 22:54 HS
(ase-gui) [g202425580@hpcmaster-1 Bulk]$
```

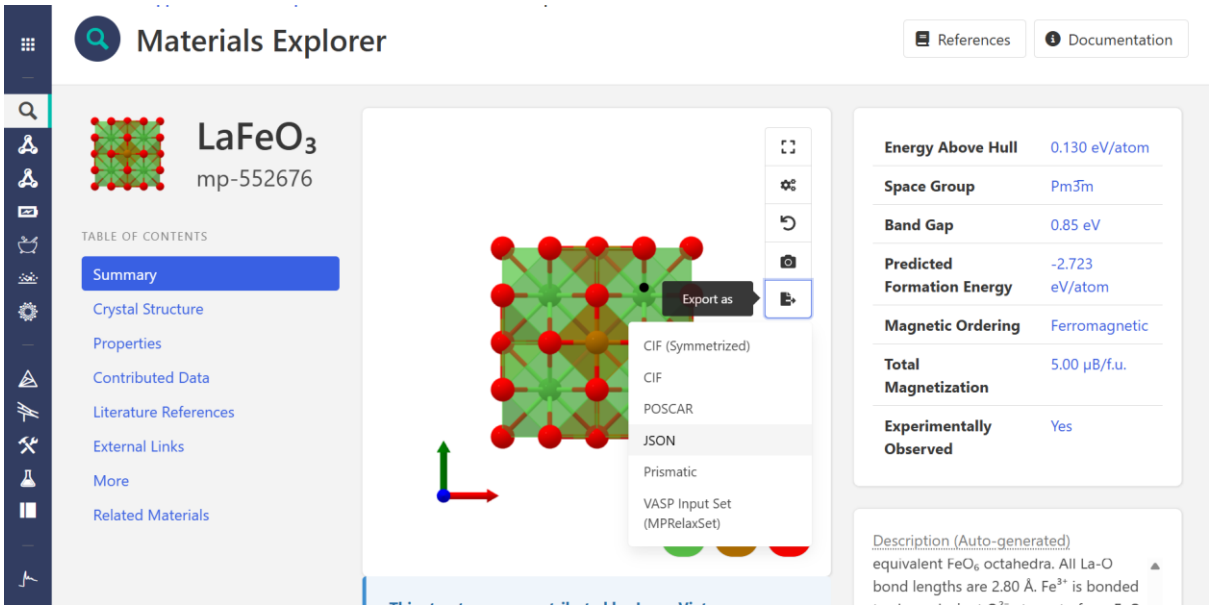
Note: High Spin (HS) and Low Spin (LS)

High Spin (HS) and Low Spin (LS) refer to different electronic configurations of transition metal ions in a crystal field.

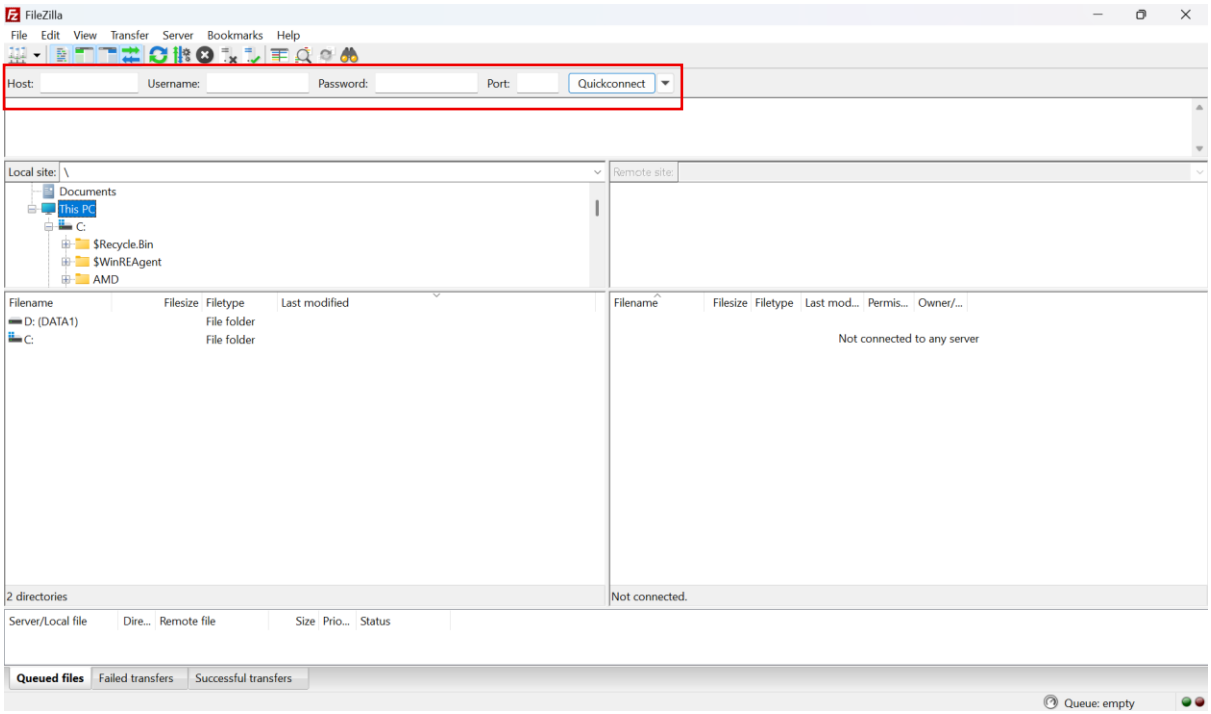
- **High Spin (HS)**:
  Electrons occupy higher energy orbitals before pairing, resulting in a **larger magnetic moment**.
  Example: Fe^{3+} (d^5) in octahedral coordination often gives ~5 μB.
- **Low Spin (LS)**:
  Electrons pair in lower energy orbitals first, resulting in a **smaller magnetic moment**.
  Example: Fe^{3+} in strong crystal fields may give ~1 μB or close to 0 μB.

In DFT calculations, setting appropriate **initial magnetic moments (initial_magmoms)** helps the system converge to the correct magnetic state.

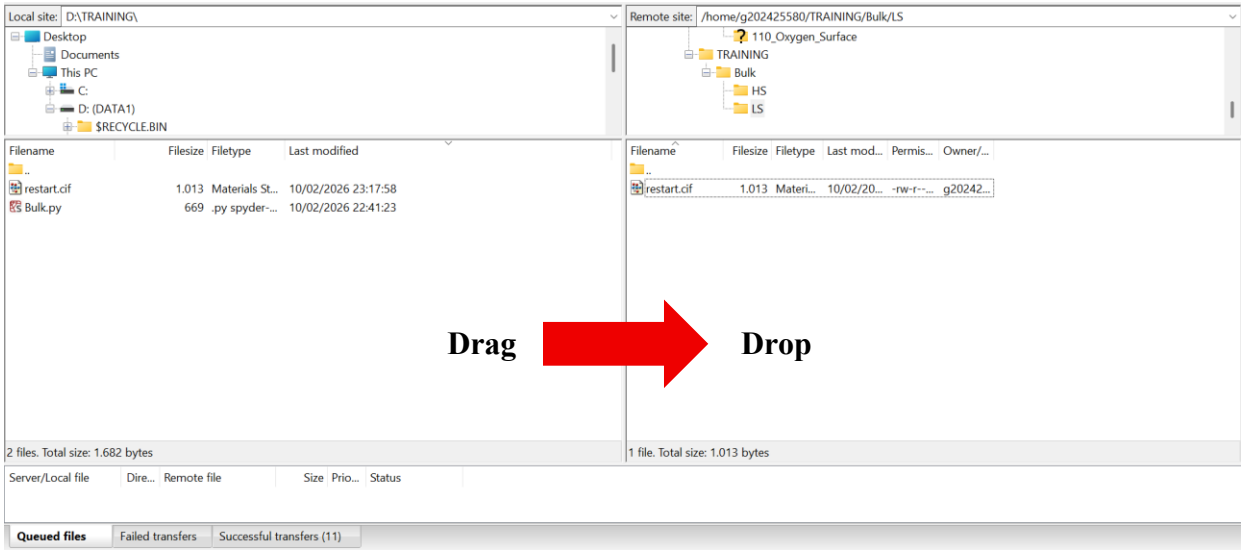
9) Prepare our initial bulk unit cell for LS, example LaFeO3. Open Materials Project in the search engine and find LaFeO3 and download **CIF** file.



10) Change the name with **restart.cif** and convert download file from our computer to server with FileZilla. Open FileZilla and enter the following details, then click **Quickconnect**: **Host:** kfupmhpc.kfupm.edu.sa, **Username:** g202xxxxxx, **Password:** your account password, and **Port:** 22.



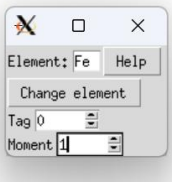
Drag and drop the required files from the left panel (your local computer) to the right panel (your directory on the HPC server) to upload them successfully.

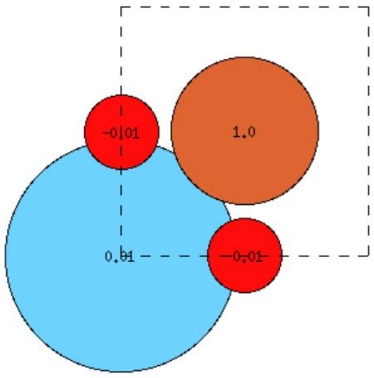


Now we have **restart.cif** for our bulk unit cell structure in the LS directory, open "**cd LS**" and check "**ls -ltr**"

```
-rw-r--r--. 1 g202425580 hpc 1013 Feb 10 23:18 restart.cif
[1]+  Done                    ase gui restart.json
(ase-gui) [g202425580@hpcmaster-1 LS]$
```

11) Open the file with ASE GUI "**ag restart.cif &**". In the ASE GUI, click **View → Show Labels → Magnetic Moments.** We can see all the magnetic moments is zero. We can change for La^{3+} with 0.01, Fe^{3+} with 1 (for LS) and 5 (for HS), for O^{2-} change with -0.01. To change the magnetic moments, select the atom and **CTRL+Y** and change the **Moment.**





12) After finish, click **file → save,** and save with the name **restart.json.** Check again the file with "**ls -ltr**"

```
-rw-r--r--. 1 g202425580 hpc 1013 Feb 10 23:18 restart.cif
-rw-r--r--. 1 g202425580 hpc  760 Feb 10 23:39 restart.json
(ase-gui) [g202425580@hpcmaster-1 LS]$
```

13) Download **run3.py** and **ase_script.slurm** in the GitHub https://github.com/jahrilnurfauzan/DFT after that transfer the file from our computer to server with FileZilla

```
(ase-gui) [g202425580@hpcmaster-1 ~]$ cd TRAINING/Bulk/LS/
(ase-gui) [g202425580@hpcmaster-1 LS]$ ls -ltr
total 10
-rw-r--r--. 1 g202425580 hpc 1013 Feb 10 23:18 restart.cif
-rw-r--r--. 1 g202425580 hpc  760 Feb 10 23:39 restart.json
-rw-r--r--. 1 g202425580 hpc 4259 Feb 11 00:04 run3.py
-rw-r--r--. 1 g202425580 hpc  780 Feb 11 00:04 ase_script.slurm
(ase-gui) [g202425580@hpcmaster-1 LS]$
```

14) Double-check each file. To open a file, use "more File_Name," for example, "**more run3.py**" Or, if you want to edit it, use "vi File_Name," for example, "**vi run3.py**" Then, press "**i**" then move to the line you want to edit and save it with "**wq!**"

15) Once you are sure, submit your slurm with "**sbatch ase_script.slurm**"

```
-rw-r--r--. 1 g202425580 hpc 1013 Feb 10 23:18 restart.cif
-rw-r--r--. 1 g202425580 hpc  760 Feb 10 23:39 restart.json
-rw-r--r--. 1 g202425580 hpc 4259 Feb 11 00:04 run3.py
-rw-r--r--. 1 g202425580 hpc  781 Feb 11 00:17 ase_script.slurm
(ase-gui) [g202425580@hpcmaster-1 LS]$ sbatch ase_script.slurm
Submitted batch job 2159454
(ase-gui) [g202425580@hpcmaster-1 LS]$
```

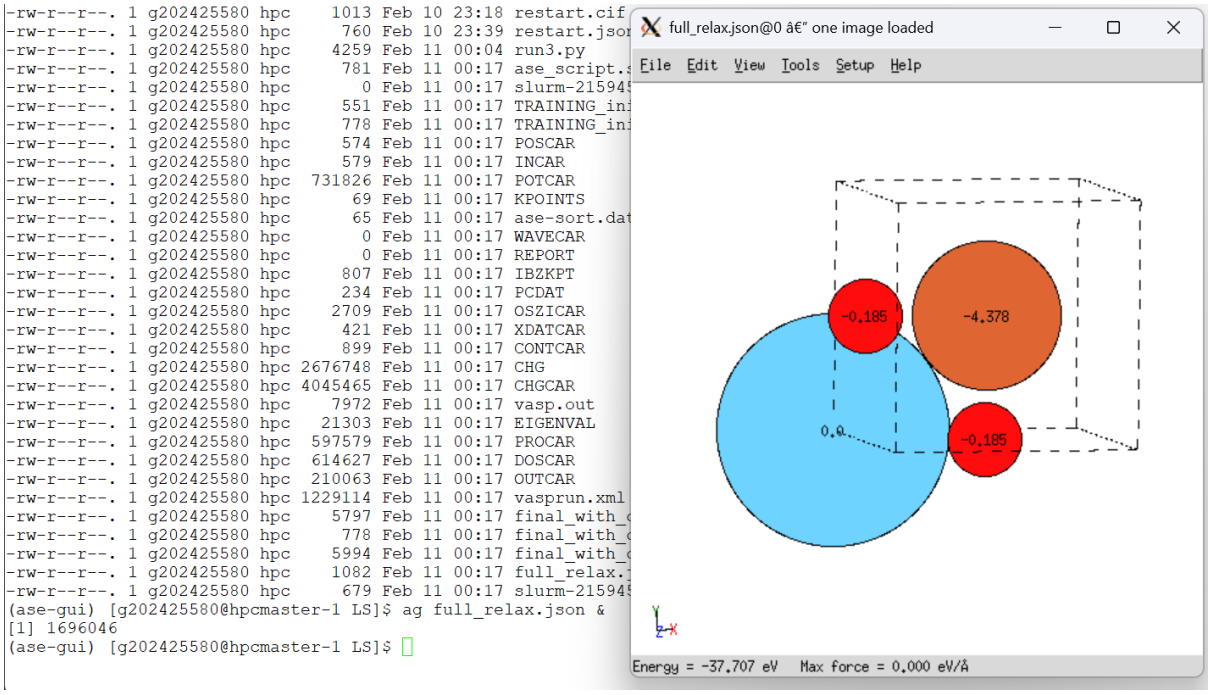
16) After submission, you will see a message like **"Submitted batch job 2159454"** (*this number is number of your job)*, indicating that the job has been successfully submitted.

17) You can then check the status of your job using the command "**me**"

18) After finish, check SLURM output files using "**more slurm-<jobID>.err**" to confirm that no runtime errors occurred, then check "**more slurm-<jobID>.out**" to review the job log.

```
-rw-r--r--. 1 g202425580 hpc    1013 Feb 10 23:18 restart.cif
-rw-r--r--. 1 g202425580 hpc     760 Feb 10 23:39 restart.json
-rw-r--r--. 1 g202425580 hpc    4259 Feb 11 00:04 run3.py
-rw-r--r--. 1 g202425580 hpc     781 Feb 11 00:17 ase_script.slurm
-rw-r--r--. 1 g202425580 hpc       0 Feb 11 00:17 slurm-2159454.err
-rw-r--r--. 1 g202425580 hpc     551 Feb 11 00:17 TRAINING_init.traj
-rw-r--r--. 1 g202425580 hpc     778 Feb 11 00:17 TRAINING_init.cif
-rw-r--r--. 1 g202425580 hpc     574 Feb 11 00:17 POSCAR
-rw-r--r--. 1 g202425580 hpc     579 Feb 11 00:17 INCAR
-rw-r--r--. 1 g202425580 hpc  731826 Feb 11 00:17 POTCAR
-rw-r--r--. 1 g202425580 hpc      69 Feb 11 00:17 KPOINTS
-rw-r--r--. 1 g202425580 hpc      65 Feb 11 00:17 ase-sort.dat
-rw-r--r--. 1 g202425580 hpc       0 Feb 11 00:17 WAVECAR
-rw-r--r--. 1 g202425580 hpc       0 Feb 11 00:17 REPORT
-rw-r--r--. 1 g202425580 hpc     807 Feb 11 00:17 IBZKPT
-rw-r--r--. 1 g202425580 hpc     234 Feb 11 00:17 PCDAT
-rw-r--r--. 1 g202425580 hpc    2709 Feb 11 00:17 OSZICAR
-rw-r--r--. 1 g202425580 hpc     421 Feb 11 00:17 XDATCAR
-rw-r--r--. 1 g202425580 hpc     899 Feb 11 00:17 CONTCAR
-rw-r--r--. 1 g202425580 hpc 2676748 Feb 11 00:17 CHG
-rw-r--r--. 1 g202425580 hpc 4045465 Feb 11 00:17 CHGCAR
-rw-r--r--. 1 g202425580 hpc    7972 Feb 11 00:17 vasp.out
-rw-r--r--. 1 g202425580 hpc   21303 Feb 11 00:17 EIGENVAL
-rw-r--r--. 1 g202425580 hpc  597579 Feb 11 00:17 PROCAR
-rw-r--r--. 1 g202425580 hpc  614627 Feb 11 00:17 DOSCAR
-rw-r--r--. 1 g202425580 hpc  210063 Feb 11 00:17 OUTCAR
-rw-r--r--. 1 g202425580 hpc 1229114 Feb 11 00:17 vasprun.xml
-rw-r--r--. 1 g202425580 hpc    5797 Feb 11 00:17 final_with_calculator.traj
-rw-r--r--. 1 g202425580 hpc     778 Feb 11 00:17 final_with_calculator.cif
-rw-r--r--. 1 g202425580 hpc    5994 Feb 11 00:17 final_with_calculator.json
-rw-r--r--. 1 g202425580 hpc    1082 Feb 11 00:17 full_relax.json
-rw-r--r--. 1 g202425580 hpc     679 Feb 11 00:17 slurm-2159454.out
(ase-gui) [g202425580@hpcmaster-1 LS]$ more slurm-2159454.err
(ase-gui) [g202425580@hpcmaster-1 LS]$
```

19) Check the magnetic moments with "**ag full_relax.json**", **View → Show Labels → Magnetic Moments**



20) Open "**vasp.out**" and check the activation energy (e0), if you see "reached required accuracy - stopping structural energy minimisation" its mean convergence

```
DAV:   6    -0.138597752070E+03    0.13292E+04   -0.16696E+03  1696   0.251E+02    0.218E+02
DAV:   7    -0.560971805670E+02    0.82501E+02   -0.40858E+02  1888   0.120E+02    0.137E+02
DAV:   8    -0.731329308086E+02   -0.17036E+02   -0.20420E+02  1968   0.232E+02    0.930E+01
DAV:   9    -0.436382637265E+02    0.29495E+02   -0.55157E+02  1808   0.712E+01    0.548E+01
DAV:  10    -0.361154528359E+02    0.75228E+01   -0.16539E+01  1968   0.472E+01    0.127E+01
DAV:  11    -0.362043466061E+02   -0.88894E-01   -0.22782E+00  1984   0.842E+00    0.101E+01
DAV:  12    -0.362771848294E+02   -0.72838E-01   -0.23981E+00  2272   0.153E+01    0.918E+00
DAV:  13    -0.366338028314E+02   -0.35662E+00   -0.70215E-01  2192   0.839E+00    0.865E+00
DAV:  14    -0.367559858693E+02   -0.12218E+00   -0.97666E-02  1520   0.234E+00    0.941E+00
DAV:  15    -0.371733098514E+02   -0.41732E+00   -0.99822E-02  1744   0.543E+00    0.558E+00
DAV:  16    -0.376789545521E+02   -0.50564E+00   -0.66439E-01  1872   0.130E+01    0.811E+00
DAV:  17    -0.376917391127E+02   -0.12785E-01   -0.11491E-01  1328   0.178E+00    0.768E+00
DAV:  18    -0.377180927643E+02   -0.26354E-01   -0.59267E-02  2032   0.155E+00    0.658E+00
DAV:  19    -0.377038623848E+02    0.14230E-01   -0.71621E-02  1824   0.205E+00    0.609E+00
DAV:  20    -0.377195303558E+02   -0.15668E-01   -0.19733E-02  1904   0.147E+00    0.310E+00
DAV:  21    -0.377058273836E+02    0.13703E-01   -0.47726E-02  1968   0.189E+00    0.173E+00
DAV:  22    -0.377068638923E+02   -0.10365E-02   -0.67027E-03  1744   0.536E-01    0.143E+00
DAV:  23    -0.377070221363E+02   -0.15824E-03   -0.22684E-02  1824   0.978E-01    0.440E-01
DAV:  24    -0.377073291210E+02   -0.30698E-03   -0.65263E-03  1872   0.448E-01    0.227E-01
DAV:  25    -0.377074136128E+02   -0.84492E-04   -0.98133E-04  1680   0.257E-01    0.250E-01
DAV:  26    -0.377073307590E+02    0.82854E-04   -0.16542E-04  1808   0.121E-01    0.185E-01
DAV:  27    -0.377073364026E+02   -0.56436E-05   -0.10441E-04  1744   0.167E-01    0.360E-02
DAV:  28    -0.377073355152E+02    0.88738E-06   -0.40592E-05  1648   0.708E-02
   1 F= -.37707336E+02 E0= -.37707336E+02  d E =-.377073E+02  mag=    -4.9989
 curvature:   0.00 expect dE= 0.000E+00 dE for cont linesearch  0.000E+00
 trial: gam= 0.00000 g(F)=  0.306E-59 g(S)=  0.000E+00 ort = 0.000E+00 (trialstep = 0.100E+01)
 search vector abs. value=  0.100E-09
 reached required accuracy - stopping structural energy minimisation
```

restart.cif
Initial bulk crystal structure in CIF format, used as the starting geometry.

restart.json
ASE-formatted JSON structure file, including atomic positions and magnetic moments, used as input for ASE–VASP runs.

run3.py
Python script that controls the VASP calculation through ASE, including relaxation settings and restart logic.

ase_script.slurm
SLURM batch submission script defining computational resources and job execution commands.

slurm-2159454.err
SLURM error file containing runtime error messages (empty if the job runs without errors).

TRAINING_init.traj
Initial atomic structure saved by ASE before starting the VASP calculation.

TRAINING_init.cif
Initial atomic structure saved in CIF format for visualization and reference.

POSCAR
VASP input file containing the initial atomic structure and lattice vectors.

INCAR
VASP input file specifying calculation parameters such as convergence criteria, smearing, and DFT+U settings.

POTCAR
Pseudopotential file containing PAW potentials for all elements in the calculation.

KPOINTS
File defining the k-point mesh used for Brillouin zone sampling.

ase-sort.dat
Internal ASE file used to keep track of atom ordering between ASE and VASP.

WAVECAR
Wavefunction file used for restarting calculations and advanced electronic structure analysis.

REPORT
Internal VASP report file, generally not required for user analysis.

IBZKPT
List of irreducible k-points generated by VASP for the calculation.

PCDAT
File related to polarization or charge-density data, depending on INCAR settings.

OSZICAR
Short summary of electronic and ionic convergence for each iteration step.

XDATCAR
Trajectory of atomic positions during ionic relaxation or molecular dynamics.

CONTCAR
Final relaxed atomic structure, in the same format as the POSCAR file.

CHG
Partial charge density file, generated depending on INCAR settings.

CHGCAR
Total charge density file, commonly used for DOS, band structure, and charge analysis.

vasp.out
Standard output of the VASP run, containing general runtime information.

EIGENVAL
File containing eigenvalues of electronic states, used for band structure calculations.

PROCAR
Projected band structure and density of states information, including orbital and atomic contributions.

DOSCAR
Density of states data generated by VASP.

OUTCAR
Main output file containing detailed information about the calculation, including energies, forces, stresses, magnetic moments, and convergence behavior.

vasprun.xml
XML file containing complete calculation data, widely used by post-processing tools such as pymatgen.

final_with_calculator.traj
Final relaxed structure stored in ASE trajectory format with calculator information.

final_with_calculator.cif
Final relaxed structure saved in CIF format for visualization.

final_with_calculator.json
Final relaxed structure saved in ASE JSON format for restart or further ASE-based calculations.

full_relax.json
ASE-generated JSON file containing the full relaxation information extracted from VASP outputs.

slurm-2159454.out
Standard output file generated by the SLURM scheduler, containing job execution logs and printed messages.

The basic HPC workflow is now complete; next, you can use the **CONTCAR** file as the relaxed structure and convert it back to a **POSCAR** file to build supercells or slabs in the ASE GUI, apply atomic constraints, further optimize INCAR settings, or even add molecules using the same workflow.

XI. Basic Directory and File Commands

1) Check the current directory

```
pwd
```

Shows your current working directory.

2) Create a new directory

```
mkdir directory_name
```

#example mkdir Project_01

3) Enter a directory

```
cd directory_name
```

#example cd Project_01

4) Go back one step

```
cd ../
```

5) Go to the home directory

```
cd ~
```

6) List files and directories

```
ls
```

And if you want to detailed view use

```
ls -ltr
```

7) Create an empty file

```
touch file_name
```

#example touch INCAR

8) Copy files or directories

```
cp source destination
```

#example cp INCAR ../Project_02 (*For example, we have an INCAR file in the Project_01 directory and we want to **copy** it to another directory such as Project_02*)

9) Move files or directories (cut)

```
mv source destination
```

#example mv INCAR ../Project_02 (*For example, we have an INCAR file in the Project_01 directory and we want to **cut** it to another directory such as Project_02*)

10) Delete files or directories

```
rm file_name
```

#example rm INCAR (*file*)

```
rmdir directory_name
```

#example rm Project_02 (*directory*)

11) Enter a file

```
more file_name
```

#example more INCAR

12) Edit and save a file

```
vi file_name
```

#example vi INCAR (*for example if you use the vi INCAR command you will open the INCAR file, then press the "**i**" button and you can edit*)

After editing, you can save the file changes by first pressing "**esc**" then pressing this button at the same time as "**shift**" and "**:**" then entering this prompt

```
wq!
```

13) Check running jobs

```
squeue -u username
```

Or we can use

```
me
```

Shows all active jobs submitted by your account.

14) Cancel a job

```
scancel jobID
```

#example scancel 2159454

Stops a running or pending SLURM job.

15) Check convergence quickly

```
tail OSZICAR
```

Shows the last electronic and ionic steps to monitor energy convergence.

16) Check electronic SCF loops

```
grep "LOOP" OUTCAR
```

Displays electronic self-consistent field (SCF) iteration timing and stability information.

17) Check total energy

```
grep "free  energy   TOTEN" OUTCAR
```

Extracts the final total energy of the system.

18) Check magnetic moments

```
grep -A 20 "magnetization" OUTCAR
```

Displays magnetic moments of each atom after convergence.

19) Check ionic convergence

```
grep "reached required accuracy" OUTCAR
```

Confirms whether the relaxation converged successfully.

20) Monitor job in real time

```
tail -f vasp.out
```

Shows live output while the calculation is running.

21) Check k-points used

```
cat KPOINTS
```

Displays the k-point mesh definition.

22) Check INCAR parameters

```
more INCAR
```

Verifies calculation settings such as ENCUT, ISMEAR, ISPIN, and LDAU.

23) Check structure

```
ase gui CONTCAR &
```

Opens the final relaxed structure in ASE GUI.

24) Copy final structure for restart

```
cp CONTCAR restart.cif
```

Uses the relaxed structure for the next calculation.

25) Search specific keywords in a file

```
grep "keyword" file_name
```

#examplegrep "EDIFF" INCAR

Searches for specific parameters or information inside a file.

26) Check disk usage

```
du -sh *
```

Shows disk space usage of files and directories.

27) Check file size

```
ls -lh
```

Lists files with human-readable sizes.

28) Convert files using ASE

```
ase convert input output
```

#example ase convert CONTCAR final.cif

Converts structure files between formats.

29) Count number of atoms

```
grep -n "" POSCAR
```

Helps verify the number of atoms in a POSCAR file.

30) Check job history

```
sacct -u username
```

Shows completed SLURM jobs and their status.